

This should output 'Hello World!' if all is working well. Next try the command given below:

```
$ node tests/hello_world.js
```

This should also show the same output: 'Hello World!'... if all is well. Now open the file `~/emscripten` (the settings file) to make a few changes. First, make the Emscripten root your current Emscripten directory:

```
JAVA='java'
EMSCRIPTEN_ROOT = os.path.expanduser('/home/user/porting/emscripten') # this helps projects using Emscripten find it
```

LLVM root: Just find in which directory `clang++` is located, and make that directory your LLVM root:

```
LLVM_ROOT = os.path.expanduser('/home/user/porting/clang-build/build/Release/bin')
```

Also make `NODE_JS='node'` in the same file. Now, try running Emscripten:

```
$ ./emcc tests/hello_world.cpp
$ node a.out.js
```

This should print 'Hello World!'. Using Emscripten, you have now transformed `hello_world.cpp` into an `a.out.js` file.

How Emscripten works

Let's take a look at the sample code in `hello_world.cpp`:

```
#include<stdio.h>

class Test {}; // This will fail in C mode

int main() {
    printf("hello, world!\n");
    return 1;
}
```

The output JavaScript code is a bit lengthy. You can find the whole code in `tests/hello_world.js` in the Emscripten directory, but here is a small part of it:

```
if (typeof print === 'undefined') {
    this['print'] = printErr;
}
// *** Environment setup code ***

print('hello, world!');
```

The way Emscripten works is that it creates LLVM bytecode

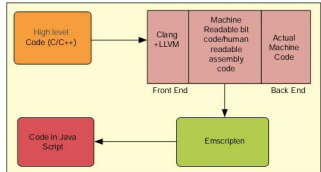


Figure 1: A flowchart explaining the role of Emscripten

files for every source file in the build process, which are then converted to JavaScript (see Figure 1). So basically, the entire code is built using `emcc`, and once built, it is converted to JS. If you pass a `.c/.cpp` file to `emcc`, it outputs a `.bc` file that contains the bytecode; and if you pass `.bc` files to it, it outputs a `.js` file.

Applications ported with Emscripten

Here is a list of a few applications ported to the Web using Emscripten.

1. **Banana Bread Engine:** Cube 2 is a 3D game engine. It is now compiled into JavaScript and WebGL, and is known as 'Banana Bread Engine'. You can run it in your Web browser itself without any plug-ins.
2. **Sql.js:** Porting of SQLite to JavaScript resulted in `sql.js`. This is obtained by compiling the SQLite C code with Emscripten.
3. **Speak.js:** It uses eSpeak, which is an open source speech synthesiser. eSpeak was compiled from C++ to JavaScript as `speak.js`. This converts text to speech.

There are many more applications compiled from C/C++ to JS using Emscripten. The list can be found at <https://github.com/kripken/emscripten/wiki>.

Most Web applications use JavaScript. The task of porting becomes much easier with Emscripten. So try it out if you need it! 

References

- [1] <https://github.com/kripken/emscripten/wiki/Tutorial>
- [2] <http://bit.ly/nlFsJ4>
- [3] <http://samcaldwel.net/index.php/technical-articles/3-how-to-articles/120-how-do-i-install-nodejs-on-ubuntu-1004>
- [4] <https://github.com/kripken/BananaBread/>
- [5] <https://github.com/kripken/sql.js>
- [6] <http://syntensity.com/static/espeak.html>
- [7] <https://github.com/kripken/emscripten/wiki>

By: Sowmya Ravidas

The author is currently pursuing her Bachelor of Technology degree in Computer Science at Amrita University, India. She has been actively contributing to open source technologies for the last one year. Currently she is working on security related issues in embedded systems. You can reach her at sowmya.ravidas@gmail.com.